

Connect Four — Updated Code

Lab 4: Vibe Coding — Scenario 15

Updated files: board.py, game.py, ai.py

board.py

ROWS, COLS = 6, 7

```
class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))

    def winner(self, token):
        # Horizontal, vertical, diagonal down-right, diagonal down-left
        directions = [
            (0, 1),
            (1, 0),
            (1, 1),
            (1, -1)
        ]

        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue

                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS
                        and 0 <= c + i * dc < COLS
                        and self.grid[r + i * dr][c + i * dc] == token
                        for i in range(4)
                    ):
                        return True

        return False

    def print(self):
        print("\n " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print(" " + " ".join(row))
```

game.py

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
                    return

                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number.")
                    continue

                if not 0 <= col < 7:
                    print("Enter a column number.")
                    continue

            else:
                col = self.ai.choose_column(self.board)

                if col is None:
                    self.board.print()
                    print("AI has no available move. Draw.")
                    return

            # Attempt the actual move
            row = self.board.drop(col, self.turn)

            if row is None:
                print("Column unavailable.")

                if self.turn == "O":
                    return

                continue

            # Task 4: feedback exactly once for each successful move
            print(self.turn, "placed in column", col + 1)

            # Check win immediately after the actual move
            if self.board.winner(self.turn):
                self.board.print()
                print(self.turn, "wins!")
                return

            # Check draw before requesting another turn
            if self.board.full():
                self.board.print()
                print("Draw.")
                return

            self.turn = "O" if self.turn == "X" else "X"
```

ai.py

```
import random

class AI:
    def choose_column(self, board, me="O", opponent="X"):
        legal = [c for c in range(7) if board.grid[0][c] == "."]

        if not legal:
            return None

        # 1. Take an immediate winning move
        for col in legal:
            row = board.drop(col, me)

            if row is not None:
                won = board.winner(me)
                board.grid[row][col] = "."

                if won:
                    return col

        # 2. Block the opponent's immediate winning move
        for col in legal:
            row = board.drop(col, opponent)

            if row is not None:
                won = board.winner(opponent)
                board.grid[row][col] = "."

                if won:
                    return col

        # 3. Otherwise choose any legal column
        return random.choice(legal)
```